

# Automated Training & Tuning with an Open-Source AutoML Tool

## LEARNING OUTCOMES

- Use an open-source AutoML tool to train and tune a model on a given dataset.
- Compare an AutoML result against a simple baseline model.
- Inspect the best model, its hyperparameters and its score.
- Save the resulting model for later use.

## Background

AutoML automates the repetitive parts of model building , choosing an algorithm and tuning its hyperparameters so you can reach a strong model quickly. In this practical you use FLAML, a lightweight open-source AutoML library, to search for a good classifier within a fixed time budget, then check whether it beats a hand-built baseline.

## Instructions

### Task 1 : Install and load data

```
pip install "flaml[automl]" scikit-learn
```

Load a dataset and split it. The example uses a built-in dataset; substitute your given dataset if provided.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

### Task 2 : Build a baseline

Always measure AutoML against something. Train a plain model first and record its accuracy.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

base = LogisticRegression(max_iter=2000).fit(X_train, y_train)
base_acc = accuracy_score(y_test, base.predict(X_test))
print('Baseline accuracy:', round(base_acc, 4))
```

### Task 3 : Run AutoML

Give FLAML a task, a metric and a time\_budget (in seconds). It will search across models and hyperparameters automatically.

```
from flaml import AutoML

automl = AutoML()
automl.fit(X_train, y_train,
          task='classification',
          metric='accuracy',
          time_budget=60)      # increase for a better search
```

#### Task 4 : Inspect the best model

```
print('Best estimator :', automl.best_estimator)
print('Best config    :', automl.best_config)
print('Best CV score  :', 1 - automl.best_loss)
```

#### Task 5 : Evaluate on the test set

```
auto_acc = accuracy_score(y_test, automl.predict(X_test))
print('AutoML test accuracy :', round(auto_acc, 4))
print('Baseline test accuracy:', round(base_acc, 4))
```

#### Task 6 : Save the best model

```
import joblib
joblib.dump(automl, 'automl_model.joblib')
print('Saved automl_model.joblib')
```

Optional: re-run with a larger time\_budget or a different metric (e.g. 'roc\_auc') and note how the result changes.

### Expected Result

- A printed comparison of baseline vs AutoML accuracy on the test set.
- The best estimator name and its chosen hyperparameters.
- A saved model file produced entirely by the AutoML search.

### Submission

- Submit your report in the format prescribed by the module tutor before the next practical class.

### Reflection Questions

Answer these in your report:

1. Did AutoML beat your baseline? By how much, and was the extra compute time worth it?
2. What did AutoML automate for you, and which decisions still required a human?
3. How does increasing the time budget affect the result and the cost?